EVALUACIÓN 02

Sistema de Gestión Quirúrgica · Módulo de Quirófano

INTEGRANTE	CURSO	FECHA
Jesus Castillo Sumire	Desarrollo de Aplicaciones Web	18 de octubre de 2026

Introducción y objetivo

El objetivo de esta evaluación es ampliar el proyecto de quirófano de la Evaluación 1 con persistencia relacional comprobable, administración de usuarios y roles, control de acceso en páginas y API, manejo global de errores y auditoría automática de las operaciones. Se conservó el flujo quirúrgico construido previamente y se incorporaron las funciones de seguridad sobre la misma base hospital_db de XAMPP.

Cómo se trabajó y cómo se logró

Se revisaron primero las entidades, controladores y servicios existentes. Después se añadieron las relaciones JPA entre Usuario y Rol, las tablas nuevas y el usuario inicial; luego se configuraron Spring Security, BCrypt, las rutas autorizadas y un aspecto AOP que guarda cada operación correcta dentro de la misma transacción. El resultado se compiló y se inició conectado al MySQL local. La verificación directa confirmó 12 tablas, los tres roles solicitados y eventos reales en la tabla de auditoría.

Herramientas y conexión

Visual Studio Code para el código; Java 17, Spring Boot 4.0.8, Spring Data JPA/Hibernate, Spring Security, Spring AOP, Thymeleaf/Bootstrap y Maven para la aplicación; XAMPP MySQL/MariaDB (motor MariaDB 10.4.32) en 127.0.0.1:3306 con la base hospital_db; y Postman para repetir las solicitudes HTTP con Basic Auth. Apache de XAMPP habilita phpMyAdmin, mientras que la aplicación Java conecta directamente con el servicio de base de datos; Apache no es el servidor de la API.

Flujo y procedimiento

Inicio de sesión → UserDetailsService → consulta de usuario y rol → BCrypt → autorización de ruta → Controller → Service → Repository/JPA → MySQL
AOP intercepta la escritura correcta → guarda usuario, fecha/hora, acción, entidad e ID en auditoria_operaciones

Procedimiento realizado: iniciar MySQL en XAMPP → ejecutar la aplicación desde el proyecto abierto → dejar que Hibernate agregue las tablas → cargar roles y administrador de forma idempotente → ingresar con un perfil → verificar rutas permitidas/denegadas y revisar las filas de hospital_db. La aplicación conserva el esquema anterior mediante **ddl-auto=update**; no se borraron tablas de la Evaluación 1.

Alcance implementado y relaciones JPA

La segunda evaluación se implementó como ampliación del mismo proyecto. La tabla usuarios guarda el rol asignado con la clave foránea rol_id; la relación se modeló con **@ManyToOne** en Usuario y **@OneToMany(mappedBy = "rol")** en Rol. En las entidades quirúrgicas existentes se mantuvieron las relaciones solicitud–paciente, médico e historia; la aprobación, evaluación, programación, quirófano y equipo conservan sus propias claves foráneas.

La relación entre roles y usuarios es el requisito nuevo. Las relaciones quirúrgicas ya existían y se preservaron; Hibernate confirmó sus claves foráneas después del arranque.

Relación Usuario → Rol

```
    @Column(name = "password_hash", nullable = false, length = 100)
    private String passwordHash;

    @Column(nullable = false)
    private boolean activo = true;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "rol_id", nullable = false)
    private Rol rol;

    public Long getIdUsuario() { return idUsuario; }
    public void setIdUsuario(Long idUsuario) { this.idUsuario = idUsuario; }
    public String getNombreUsuario() { return nombreUsuario; }
    public void setNombreUsuario(String nombreUsuario) { this.nombreUsuario = nombreUsuario; }
    public String getCorreo() { return correo; }
    public void setCorreo(String correo) { this.correo = correo; }
    public String getNombres() { return nombres; }
    public void setNombres(String nombres) { this.nombres = nombres; }
    public String getApellidos() { return apellidos; }
    public void setApellidos(String apellidos) { this.apellidos = apellidos; }
    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
    public boolean isActivo() { return activo; }
```

Código nuevo: Usuario asocia cada cuenta con un Rol activo mediante rol_id.

Relación inversa Rol → Usuarios

```
    private String descripcion;

    @Column(nullable = false)
    private boolean activo = true;

    @JsonIgnore
    @OneToMany(mappedBy = "rol", fetch = FetchType.LAZY)
    private List<Usuario> usuarios = new ArrayList<>();

    public Long getIdRol() { return idRol; }
    public void setIdRol(Long idRol) { this.idRol = idRol; }
    public String getNombreRol() { return nombreRol; }
    public void setNombreRol(String nombreRol) { this.nombreRol = nombreRol; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
    public boolean isActivo() { return activo; }
    public void setActivo(boolean activo) { this.activo = activo; }
    public List<Usuario> getUsuarios() { return usuarios; }
}
```

Código nuevo: Rol expone la relación inversa con sus usuarios.

Autenticación y autorización por perfiles

El inicio de sesión web utiliza una cuenta persistida en MySQL; Postman puede autenticarse con Basic Auth. BCryptPasswordEncoder compara las contraseñas con el hash guardado. Se usa una consulta con @EntityGraph para cargar el rol junto con el usuario y evitar que la asociación diferida falle al autenticar.

La cadena de seguridad protege en el servidor, además de ocultar opciones de interfaz. ADMINISTRADOR administra cuentas, roles y auditoría; MEDICO accede a evaluación, aprobación y programación; RECEPCIONISTA registra y consulta solicitudes. Las rutas de gestión de usuarios y roles quedan restringidas al administrador.

Cadena de seguridad y rutas por rol

```
@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http, RoleLoginSuccessHandler loginSuccessHandler)
            throws Exception {
        http
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/", "/login", "/error", "/css/**", "/images/**").permitAll()
                .requestMatchers("/web/admin/**", "/api/usuarios/**", "/api/roles/**", "/api/auditoria/**")
                    .hasRole("ADMINISTRADOR")
                .requestMatchers("/web/solicitudes/*/evaluacion", "/web/solicitudes/*/decision",
                        "/web/solicitudes/*/programar", "/web/solicitudes/programaciones/**",
                        "/web/solicitudes/quirofanos/**", "/api/solicitudes/*/aprobaciones/**",
                        "/api/quirofanos/**")
                    .hasAnyRole("ADMINISTRADOR", "MEDICO")
                .requestMatchers("/api/evaluaciones-preoperatorias/**", "/api/programaciones/**")
                    .hasAnyRole("ADMINISTRADOR", "MEDICO")
                .requestMatchers("/web/solicitudes/**", "/api/solicitudes/**")
                    .hasAnyRole("ADMINISTRADOR", "MEDICO", "RECEPCIONISTA")
                .anyRequest().authenticated())
            .csrf(csrf -> csrf.ignoringRequestMatchers("/api/**"))
            .formLogin(form -> form
                .loginPage("/login")
                .successHandler(loginSuccessHandler)
                .permitAll())
            .httpBasic(Customizer.withDefaults())
            .logout(logout -> logout
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll())
            .exceptionHandling(errors -> errors.accessDeniedPage("/acceso-denegado"));
        return http.build();
    }

    @Bean
    UserDetailsService userDetailsService(UsuarioRepository usuarioRepository) {
        return nombreUsuario -> usuarioRepository.findWithRolByNombreUsuarioIgnoreCase(nombreUsuario)
                .map(usuario -> User.withUsername(usuario.getNombreUsuario())
                        .password(usuario.getPasswordHash())
                        .roles(usuario.getRol().getNombreRol())
                        .disabled(!usuario.isActivo() || !usuario.getRol().isActivo())
```

Código nuevo: SecurityFilterChain, perfiles, ingreso con formulario/Basic Auth y carga del rol.

Administrador local inicial: usuario admin y contraseña Admin123!. El código la cifra con BCrypt antes de almacenarla. Es una credencial de laboratorio; debe cambiarse antes de cualquier despliegue real.

Auditoría transversal con AOP

La anotación @AuditOperation identifica acción y entidad en los métodos que escriben. AuditAspect se ejecuta después de un resultado exitoso, obtiene el usuario autenticado, toma el ID de la entidad devuelta o de los argumentos y envía el evento a AuditoriaService. La auditoría usa propagación obligatoria: si la operación principal falla, no queda un evento engañoso; cuando tiene éxito, el evento se guarda en la misma transacción.

Auditoría automática AOP

```
import com.hospital.quirofano.entity.AuditoriaOperacion;
import com.hospital.quirofano.service.AuditoriaService;

@Aspect
@Component
@Order(Ordered.LOWEST_PRECEDENCE)
public class AuditAspect {

    private final AuditoriaService auditoriaService;

    public AuditAspect(AuditoriaService auditoriaService) {
        this.auditoriaService = auditoriaService;
    }

    @AfterReturning(pointcut = "@annotation(operation)", returning = "result")
    public void registrarOperacion(JoinPoint joinPoint, AuditOperation operation, Object result) {
        AuditoriaOperacion auditoria = new AuditoriaOperacion();
        auditoria.setUsuario(usuarioActual());
        auditoria.setOperacion(operation.action());
        auditoria.setEntidad(operation.entity());
        auditoria.setRegistroId(identificador(result, joinPoint.getArgs()));
        auditoria.setDetalle(joinPoint.getSignature().getName());
        auditoriaService.registrar(auditoria);
    }

    private String usuarioActual() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        if (authentication == null || !authentication.isAuthenticated()
                || authentication instanceof AnonymousAuthenticationToken) {
            return "SISTEMA";
        }
        return authentication.getName();
    }

    private String identificador(Object resultado, Object[] argumentos) {
        String idResultado = buscarId(resultado);
        if (idResultado != null) return idResultado;
        return Arrays.stream(argumentos)
                .map(this::buscarId)
                .filter(id -> id != null)
                .findFirst()
                .orElse(null);
    }

    private String buscarId(Object valor) {
        if (valor == null) return null;
        if (valor instanceof ResponseEntity<?> response) return buscarId(response.getBody());
        if (valor instanceof Number number) return number.toString();

        for (Method method : valor.getClass().getMethods()) {
            String nombre = method.getName();
            if (method.getParameterCount() != 0 || !Number.class.isAssignableFrom(method.getReturnType())) continue;
            boolean esPropiedadId = nombre.startsWith("getId")
                    || nombre.matches("id[A-Z].*");
            if (!esPropiedadId) continue;
            try {
```

Código nuevo: aspecto que registra usuario, acción, entidad, identificador y método ejecutado.

Se auditan altas, modificaciones, cambios de estado y cancelaciones de solicitudes; evaluación y decisión clínica; programación, reprogramación y cancelación; mantenimiento de quirófanos; usuarios y roles. La vista administrativa muestra hasta 200 eventos recientes. La fecha se persiste en hora local de Lima mediante configuración de zona horaria JDBC.

Auditoría conectada al flujo de quirófano

La auditoría se integra en los servicios que ya ejecutan las reglas de negocio. Por ejemplo, al registrar una evaluación, aprobar una solicitud o programar una cirugía, el servicio conserva las validaciones previas y AOP añade la fila correspondiente en auditoria_operaciones. No se reemplazaron las reglas de autorización clínica ni de disponibilidad de salas.

Auditoría conectada al proceso quirúrgico

```
    return evaluacionRepository.findAll();
}

public EvaluacionPreoperatoria buscarEvaluacion(Long id) {
    return evaluacionRepository.findById(id).orElseThrow(() -> noEncontrado("Evaluación preoperatoria", id));
}

@Transactional
@AuditOperation(action = "CREAR", entity = "evaluaciones_preoperatorias")
public EvaluacionPreoperatoria registrarEvaluacion(EvaluacionRequest datos) {
    validarEvaluacion(datos);
    SolicitudCirugia solicitud = buscarSolicitud(datos.solicitudId());
    if ("CANCELADA".equalsIgnoreCase(solicitud.getEstado())) {
        throw new ResponseStatusException(HttpStatus.CONFLICT, "No se puede evaluar una solicitud cancelada.");
    }
    boolean tieneProgramacionActiva = programacionRepository
            .findBySolicitud_IdSolicitudOrderByFechaHoraInicio(solicitud.getIdSolicitud()).stream()
            .anyMatch(p -> !"CANCELADA".equalsIgnoreCase(p.getEstado()));
    if (tieneProgramacionActiva) {
        throw new ResponseStatusException(HttpStatus.CONFLICT, "Cancele la programación antes de registrar otra evaluación.");
    }
    EvaluacionPreoperatoria evaluacion = new EvaluacionPreoperatoria();
    copiarEvaluacion(evaluacion, datos, solicitud);
    evaluacion.setFechaEvaluacion(LocalDateTime.now());
    solicitud.setEstado("EN_EVALUACION");
    solicitudRepository.save(solicitud);
    return evaluacionRepository.save(evaluacion);
}

@Transactional
@AuditOperation(action = "MODIFICAR", entity = "evaluaciones_preoperatorias")
public EvaluacionPreoperatoria actualizarEvaluacion(Long id, EvaluacionRequest datos) {
    validarEvaluacion(datos);
    EvaluacionPreoperatoria evaluacion = buscarEvaluacion(id);
    SolicitudCirugia solicitud = datos.solicitudId() == null ? evaluacion.getSolicitud() : buscarSolicitud(datos.solicitudId());
    boolean tieneProgramacionActiva = programacionRepository
            .findBySolicitud_IdSolicitudOrderByFechaHoraInicio(solicitud.getIdSolicitud()).stream()
            .anyMatch(p -> !"CANCELADA".equalsIgnoreCase(p.getEstado()));
    if (tieneProgramacionActiva) {
        throw new ResponseStatusException(HttpStatus.CONFLICT, "Cancele la programación antes de modificar la evaluación.");
    }
    copiarEvaluacion(evaluacion, datos, solicitud);
    evaluacion.setFechaEvaluacion(LocalDateTime.now());
    solicitud.setEstado("EN_EVALUACION");
    solicitudRepository.save(solicitud);
    return evaluacionRepository.save(evaluacion);
}

@Transactional
@AuditOperation(action = "REGISTRAR_DECISION", entity = "aprobaciones_cirugia")
public AprobacionCirugia registrarDecision(Long solicitudId, AprobacionRequest datos) {
    if (datos == null || textoVacio(datos.estado()) || textoVacio(datos.responsable())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Indique estado y responsable de la decisión.");
```

Código del servicio quirúrgico: operaciones con transacción y anotación de auditoría.

El log registra el nombre del usuario como texto y no como FK. Así queda trazabilidad aun cuando una cuenta se desactive; el campo entidad + registro_id identifica la fila afectada.

Administración de usuarios y roles

Se añadieron servicios y controladores REST/web para crear, listar, editar y activar/desactivar usuarios y roles. Las claves se actualizan solo cuando se proporciona una nueva contraseña; las respuestas REST usan DTO y no exponen password_hash. Se impide desactivar el último administrador activo y desactivar un rol que todavía tenga usuarios activos.

CRUD, BCrypt y reglas de usuario

```
public List<Usuario> listarTodos() {
    return usuarioRepository.findAll();
}

@Transactional(readOnly = true)
public Usuario buscar(Long id) {
    return usuarioRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Usuario no encontrado."));
}

@Transactional
@AuditOperation(action = "CREAR", entity = "usuarios")
public Usuario crear(UsuarioForm form) {
    validarFormulario(form, true);
    String username = normalizarUsuario(form.getNombreUsuario());
    if (usuarioRepository.existsByNombreUsuarioIgnoreCase(username)) {
        throw new ResponseStatusException(HttpStatus.CONFLICT, "El nombre de usuario ya está registrado.");
    }
    if (usuarioRepository.existsByCorreoIgnoreCase(form.getCorreo().trim())) {
        throw new ResponseStatusException(HttpStatus.CONFLICT, "El correo ya está registrado.");
    }

    Usuario usuario = new Usuario();
    copiarDatos(usuario, form, true);
    return usuarioRepository.save(usuario);
}

@Transactional
@AuditOperation(action = "MODIFICAR", entity = "usuarios")
public Usuario actualizar(Long id, UsuarioForm form) {
    Usuario usuario = buscar(id);
    validarFormulario(form, false);
    String username = normalizarUsuario(form.getNombreUsuario());
    usuarioRepository.findByNombreUsuarioIgnoreCase(username)
            .filter(otro -> !otro.getIdUsuario().equals(id))
            .ifPresent(otro -> { throw new ResponseStatusException(HttpStatus.CONFLICT, "El nombre de usuario ya está registrado."); });
    usuarioRepository.findByCorreoIgnoreCase(form.getCorreo().trim())
            .filter(otro -> !otro.getIdUsuario().equals(id))
            .ifPresent(otro -> { throw new ResponseStatusException(HttpStatus.CONFLICT, "El correo ya está registrado."); });

    copiarDatos(usuario, form, false);
    return usuarioRepository.save(usuario);
}

@Transactional
@AuditOperation(action = "CAMBIAR_ESTADO", entity = "usuarios")
public Usuario cambiarEstado(Long id) {
    Usuario usuario = buscar(id);
    if (usuario.isActivo() && "ADMINISTRADOR".equals(usuario.getRol().getNombreRol())) {
        long administradoresActivos = usuarioRepository.countByRol_NombreRolAndActivoTrue("ADMINISTRADOR");
        if (administradoresActivos <= 1) {
            throw new ResponseStatusException(HttpStatus.CONFLICT,
                    "Debe conservarse al menos un usuario administrador activo.");
        }
    }
    usuario.setActivo(!usuario.isActivo());
    return usuarioRepository.save(usuario);
}

private void copiarDatos(Usuario usuario, UsuarioForm form, boolean nuevo) {
    Rol rol = rolRepository.findById(form.getRolId())
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Rol no encontrado."));
    if (!rol.isActivo()) {
```

Código nuevo: validaciones, unicidad, asignación de rol, BCrypt y activación de cuenta.

Las páginas nuevas son /login, /web/admin/usuarios, /web/admin/roles y /web/admin/auditoria. El ingreso de un administrador abre la administración de usuarios; médicos y recepción van al panel quirúrgico. La interfaz muestra opciones según el perfil, mientras SecurityFilterChain aplica la restricción real en el backend.

Endpoints administrativos y API

API administrativa

```
        this.roles = roles;
        this.auditoria = auditoria;
    }

    @GetMapping("/usuarios")
    public List<UsuarioResponse> usuarios() { return usuarios.listarTodos().stream().map(UsuarioResponse::desde).toList(); }

    @PostMapping("/usuarios")
    @ResponseStatus(HttpStatus.CREATED)
    public UsuarioResponse crearUsuario(@Valid @RequestBody UsuarioForm form) { return UsuarioResponse.desde(usuarios.crear(form)); }

    @PutMapping("/usuarios/{id}")
    public UsuarioResponse editarUsuario(@PathVariable Long id, @Valid @RequestBody UsuarioForm form) {
        return UsuarioResponse.desde(usuarios.actualizar(id, form));
    }

    @PutMapping("/usuarios/{id}/estado")
    public UsuarioResponse cambiarEstadoUsuario(@PathVariable Long id) { return UsuarioResponse.desde(usuarios.cambiarEstado(id)); }

    @GetMapping("/roles")
    public List<RolResponse> roles() { return roles.listarTodos().stream().map(RolResponse::desde).toList(); }

    @PostMapping("/roles")
    @ResponseStatus(HttpStatus.CREATED)
    public RolResponse crearRol(@Valid @RequestBody RolForm form) { return RolResponse.desde(roles.crear(form)); }

    @PutMapping("/roles/{id}")
    public RolResponse editarRol(@PathVariable Long id, @Valid @RequestBody RolForm form) {
        return RolResponse.desde(roles.actualizar(id, form));
    }

    @PutMapping("/roles/{id}/estado")
    public RolResponse cambiarEstadoRol(@PathVariable Long id) { return RolResponse.desde(roles.cambiarEstado(id)); }

    @GetMapping("/auditoria")
    public List<AuditoriaOperacion> auditoria() { return auditoria.listarRecientes(); }
}
```

Código nuevo: endpoints administrativos con DTOs para no serializar entidades ni contraseñas.

GET / POST	/api/usuarios	Listar / crear cuenta
PUT	/api/usuarios/{id}	Editar datos
PUT	/api/usuarios/{id}/estado	Activar / desactivar
GET / POST	/api/roles	Listar / crear rol
PUT	/api/roles/{id}	Editar rol
PUT	/api/roles/{id}/estado	Activar / desactivar
GET	/api/auditoria	Ver últimos 200 eventos

Los endpoints existentes de solicitudes, evaluación, decisiones, quirófanos y programación continúan disponibles bajo sus rutas anteriores, con permisos asociados al perfil. El SQL aditivo de referencia se encuentra en database/evaluacion-02-seguridad-auditoria.sql; el esquema en ejecución lo actualiza Hibernate.

Validación y manejo global de errores REST

Se centralizaron las excepciones de los controladores REST en un único advice. Así la API entrega errores en JSON con fecha/hora, estado HTTP, mensaje y ruta, en vez de repetir respuestas distintas en cada controlador. Se distinguen datos inválidos, JSON mal formado, referencias inexistentes, conflictos de integridad, falta de permisos y errores inesperados.

Manejo global de errores REST

```
import org.springframework.web.server.ResponseStatusException;

import jakarta.servlet.http.HttpServletRequest;

@RestControllerAdvice(annotations = RestController.class)
public class ApiExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<ApiError> manejarEstado(ResponseStatusException exception, HttpServletRequest request) {
        HttpStatusCode status = exception.getStatusCode();
        String message = exception.getReason() == null ? "La solicitud no pudo procesarse." : exception.getReason();
        return respuesta(status, message, request);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> manejarValidacion(MethodArgumentNotValidException exception, HttpServletRequest request) {
        FieldError error = exception.getBindingResult().getFieldErrors().stream().findFirst().orElse(null);
        String message = error == null ? "Revise los datos enviados." : error.getField() + ": " + error.getDefaultMessage();
        return respuesta(HttpStatus.BAD_REQUEST, message, request);
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ApiError> manejarJson(HttpMessageNotReadableException exception, HttpServletRequest request) {
        return respuesta(HttpStatus.BAD_REQUEST, "El cuerpo JSON está vacío o tiene un formato no válido.", request);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ApiError> manejarIntegridad(DataIntegrityViolationException exception, HttpServletRequest request) {
        return respuesta(HttpStatus.CONFLICT, "La operación incumple una restricción de integridad o un dato único.", request);
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ApiError> manejarAcceso(AccessDeniedException exception, HttpServletRequest request) {
        return respuesta(HttpStatus.FORBIDDEN, "El perfil autenticado no tiene permiso para este recurso.", request);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> manejarErrorInterno(Exception exception, HttpServletRequest request) {
        log.error("Error inesperado atendiendo {} {}", request.getMethod(), request.getRequestURI(), exception);
        return respuesta(HttpStatus.INTERNAL_SERVER_ERROR, "Ocurrió un error interno al procesar la solicitud.", request);
    }

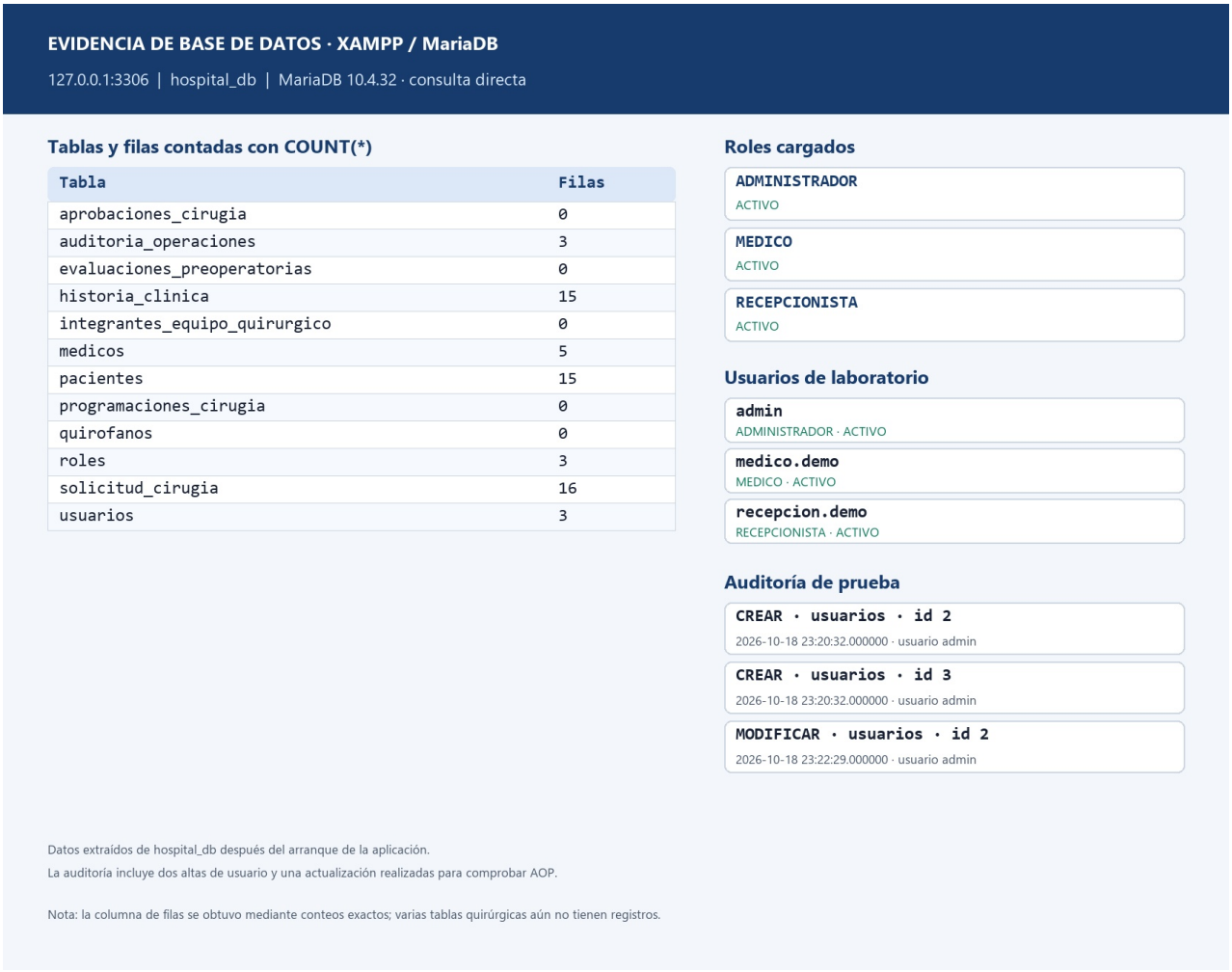
    private ResponseEntity<ApiError> respuesta(HttpStatusCode status, String message, HttpServletRequest request) {
        ApiError body = new ApiError(Instant.now(), status.value(), message, request.getRequestURI());
        return ResponseEntity.status(status).body(body);
    }
```

Código nuevo: manejador global REST con estados HTTP y mensajes seguros.

Códigos manejados: 400 para validación/formato; 401 para credenciales no válidas; 403 para rol sin permiso; 404 para recursos no encontrados; 409 para duplicados o restricciones; 500 para errores inesperados. El mensaje de error interno no expone datos técnicos al cliente.

Base de datos hospital_db en XAMPP

La siguiente imagen se generó a partir de consultas directas a MariaDB de XAMPP en 127.0.0.1:3306 (conteos exactos con COUNT(*), roles, usuarios y eventos). XAMPP presenta el servicio como MySQL en su panel; el motor identificado por SELECT VERSION() es MariaDB 10.4.32.



Evidencia de datos reales consultados en hospital_db; las contraseñas no se consultaron ni se mostraron.

El proyecto conserva 15 pacientes, 5 médicos y 15 historias clínicas; en la consulta de esta entrega había 16 solicitudes. Se añadieron las tablas de seguridad (roles, usuarios y auditoria_operaciones) y las tablas quirúrgicas complementarias que el proyecto ya mapeaba. Las tablas de evaluación, aprobación, quirófano, programación y equipo aún aparecen sin filas porque no se creó una cirugía de demostración.

Claves foráneas verificadas

La consulta a information_schema.KEY_COLUMN_USAGE devolvió 13 claves foráneas en hospital_db. La imagen presenta los pares tabla/columna y tabla/clave primaria a los que apuntan. El nuevo vínculo del requisito de usuarios aparece como usuarios.rol_id → roles.id_rol.

Relaciones verificadas en MySQL

Lectura de information_schema.KEY_COLUMN_USAGE · hospital_db

Tabla	Columna FK	Tabla referenciada	Clave PK
aprobaciones_cirugia	solicitud_id	solicitud_cirugia	id_solicitud
evaluaciones_preoperatorias	profesional_id	medicos	id_medico
evaluaciones_preoperatorias	solicitud_id	solicitud_cirugia	id_solicitud
integrantes_equipo_quirurgico	medico_id	medicos	id_medico
integrantes_equipo_quirurgico	programacion_id	programaciones_cirugia	id_programacion
programaciones_cirugia	anestesiologo_id	medicos	id_medico
programaciones_cirugia	cirujano_id	medicos	id_medico
programaciones_cirugia	quirofano_id	quirofanos	id_quirofano
programaciones_cirugia	solicitud_id	solicitud_cirugia	id_solicitud
solicitud_cirugia	historia_clinica_id	historia_clinica	id_historia
solicitud_cirugia	medico_solicitante_id	medicos	id_medico
solicitud_cirugia	paciente_id	pacientes	id_paciente
usuarios	rol_id	roles	id_rol

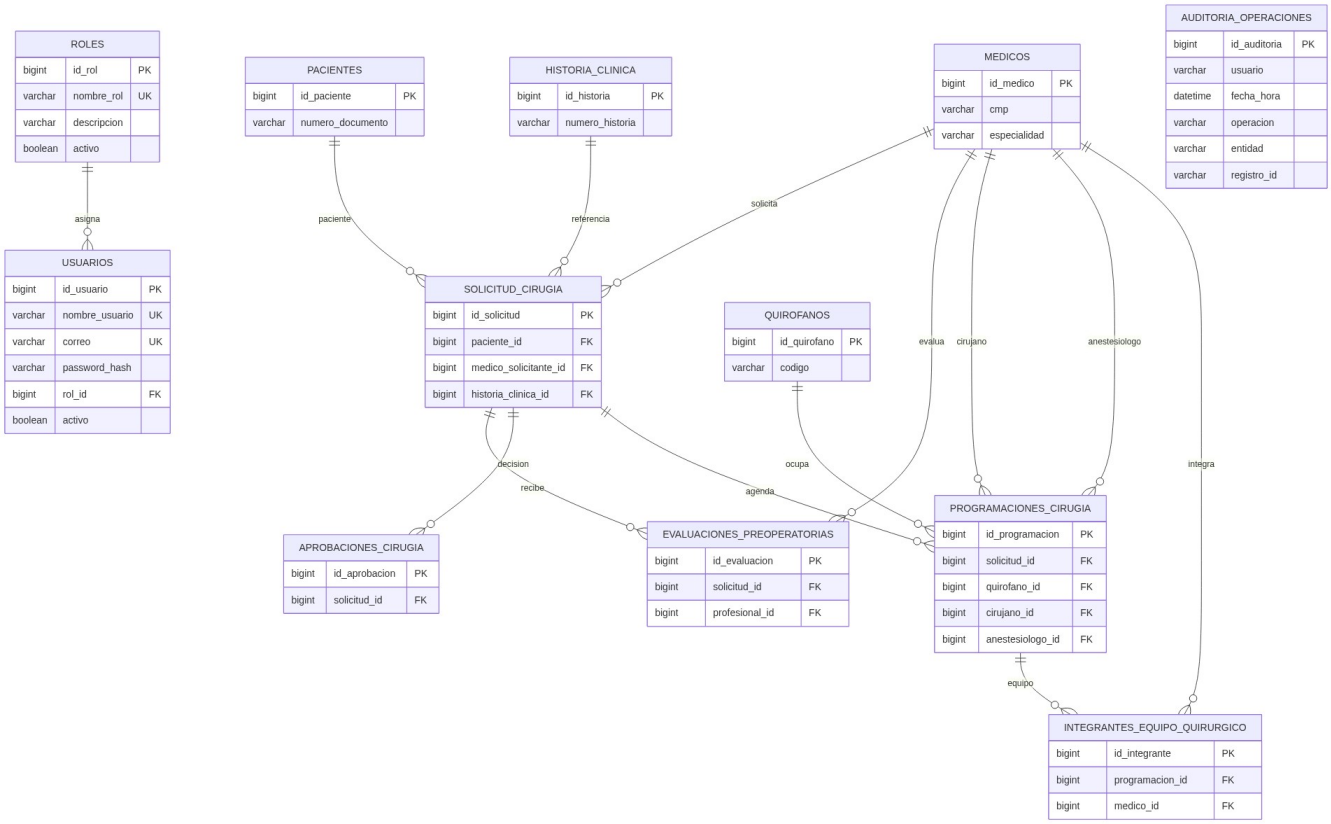
Total comprobado en la base: 13 claves foráneas.

Consulta directa de relaciones del esquema después de la actualización Hibernate.

Las relaciones quirúrgicas evitan referencias inexistentes en solicitudes, evaluaciones, aprobaciones, programación y equipo. La relación de auditoría con el usuario se mantiene por su nombre y no como FK, para que el registro histórico no dependa de la vida de la cuenta.

Diagrama final de relaciones

El diagrama se definió en Mermaid como erDiagram y se incluyó también su fuente en docs/evaluacion-02/modelo-relacional.mmd. Representa las 13 relaciones consultadas en MySQL; AUDITORIA_OPERACIONES aparece sin línea a USUARIOS porque guarda el nombre del actor como texto.



Modelo de entidades JPA y claves foráneas comprobadas en la base de datos local.

Flujo principal conservado: paciente/médico/historia → solicitud quirúrgica → evaluación preoperatoria y aprobación → programación en quirófano con profesionales → integrantes del equipo. El flujo de seguridad enlaza roles con usuarios y deja cada operación en la bitácora.

Comprobaciones y conclusión

Después de compilar e iniciar el proyecto se verificaron credenciales, vistas administrativas, serialización segura y autorización efectiva por perfil. La tabla resume respuestas observadas en solicitudes HTTP locales; se pueden repetir en Postman con Authorization → Basic Auth.

Comprobación de autenticación y permisos

Solicitudes HTTP realizadas contra la aplicación local en localhost:8080

Perfil	Recurso	HTTP	Resultado observado
Administrador	GET /api/usuarios	200	Administración disponible
Médico	GET /api/programaciones	200	Puede consultar programación
Médico	GET /api/usuarios	403	Administración denegada
Recepcionista	GET /api/solicitudes	200	Puede gestionar solicitudes
Recepcionista	GET /api/programaciones	403	Programación denegada
Administrador	POST /api/usuarios (datos inválidos	400	Validación centralizada en JSON
Administrador	GET /web/admin/usuarios	200	Pantalla administrativa disponible
Administrador	GET /web/admin/roles	200	Pantalla de roles disponible
Administrador	GET /web/admin/auditoria	200	Auditoría disponible

Comprobación local de permisos: rutas aceptadas responden 200 y rutas sin permiso responden 403.

Cuentas locales para repetir los ejemplos

admin	ADMINISTRADOR	Admin123! (inicial)
medico.demo	MEDICO	MedicoDemo2026!
recepcion.demo	RECEPCIONISTA	RecepcionDemo2026!

> Estas cuentas se dejaron activas en la base local para que se pueda reproducir la matriz de permisos. No reutilizar estas claves fuera del laboratorio; el administrador puede cambiarlas desde el panel.

Conclusión

Con esta evaluación completé una ampliación importante del sistema quirúrgico. Ahora las cuentas se relacionan con roles mediante JPA, las contraseñas se guardan con BCrypt y el backend valida qué puede hacer cada perfil. También quedó una bitácora transversal que muestra quién ejecutó la operación, qué registro afectó y cuándo ocurrió, sin llenar cada controlador con código repetido.

La integración se realizó sobre la misma base hospital_db y mantuvo las tablas y relaciones que ya existían. La aplicación compiló y arrancó conectada a MySQL de XAMPP; las tablas, los roles, usuarios y claves foráneas se verificaron directamente, y las rutas protegidas respondieron según los permisos esperados. De esta manera la Evaluación 2 suma seguridad y trazabilidad al flujo de solicitud, evaluación y programación desarrollado en la Evaluación 1.